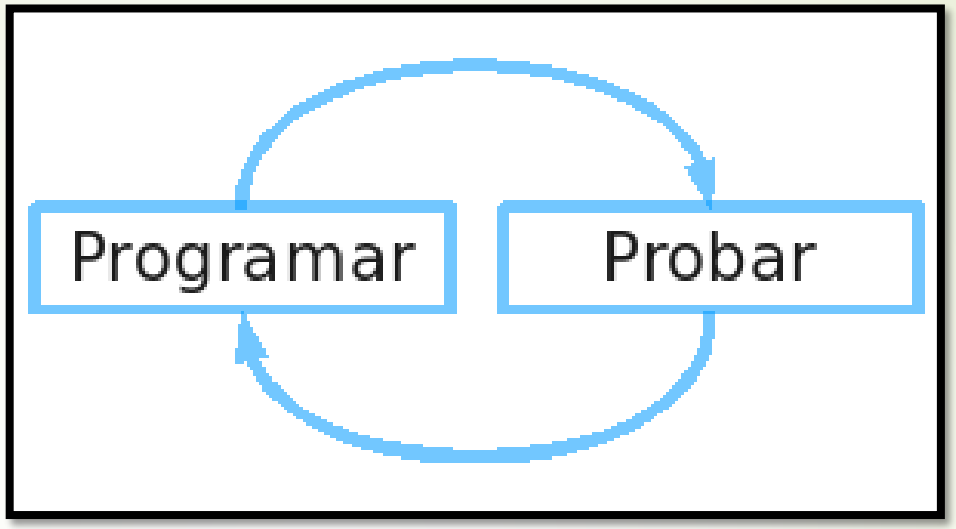
Desarrollo de Aplicaciones Web y Multiplataforma: **Entornos de Desarrollo**

DOCENTE: Daniel López Lozano

atlántida
CIDEP
Centro de Investigación y Desarrollo de Estudios Profesionales



Tema 3.
Pruebas de Software

Índice de contenidos

- **Conceptos básicos.**
- **Estrategias de prueba**
  - Pruebas unitarias.
  - Pruebas de integración.
  - Pruebas de sistema.
  - Pruebas de aceptación.
- **Técnicas de prueba.**
  - Pruebas de caja negra.
  - Pruebas de caja blanca.
- **Realizar pruebas con JUnit.**

Objetivos de las pruebas

- Una vez generado el código fuente, el software debe ser probado para descubrir el máximo posible de errores antes de la entrega al cliente.
- Nuestro objetivo es diseñar una serie de casos de prueba que tengan una alta probabilidad de encontrar errores.
- Una prueba tiene éxito si descubre un error no detectado hasta entonces.
- Nuestro objetivo es diseñar pruebas que sistemáticamente saquen a la luz diferentes clases de errores, haciéndolo con la menor cantidad de tiempo y de esfuerzo.

- Las pruebas son uno de los pasos de la ingeniería del software que se puede ver como destructivo en lugar de constructivo.
- ¿Deberíamos sentirnos culpables porque es un proceso de búsqueda de fallos?

- Existe un mito que dice que si fuéramos realmente buenos programando, no habría errores que buscar.
- Si tan sólo fuéramos realmente capaces de concentrarnos.
- Si todo el mundo empleara técnicas de codificación estructuradas, el diseño descendente, las tablas de decisión.
- Si los programas se escribieran en un lenguaje apropiado.
- Si tuviéramos siempre la solución más adecuada, entonces no habría errores. Así es el mito.

- Según el mito, existen errores porque somos malos en lo que hacemos, y si somos malos en lo que hacemos, deberíamos sentirnos culpables por ello.
- Por tanto, las pruebas, con el diseño de casos de prueba, son un reconocimiento de nuestros fallos, lo que implica una buena dosis de culpabilidad.
- Y lo tediosas que son las pruebas son un justo castigo a nuestros errores. ¿Castigados por qué? ¿Por ser humanos? ¿Culpables por qué? ¿Por no conseguir una perfección inhumana?

Extractos de Boris Beizer de su libro Software Testing Techniques

- Las pruebas deberían planificarse mucho antes de que empiecen.
- La planificación de las pruebas pueden empezar tan pronto como esté completo el modelo de requisitos.
- La definición detallada de los casos de prueba puede empezar tan pronto como el modelo de diseño se ha consolidado.
- Por tanto, se pueden planificar y diseñar todas las pruebas antes de generar ningún código.
- Las pruebas deberían empezar por «lo pequeño» y progresar hacia «lo grande».

- Las primeras pruebas planeadas y ejecutadas se centran generalmente en módulos individuales del programa.
- A medida que avanzan las pruebas, desplazan su punto de mira en un intento de encontrar errores en grupos integrados de módulos y finalmente en el sistema entero.
- No son posibles las pruebas exhaustivas. El número de permutaciones de caminos para incluso un programa de tamaño moderado es excepcionalmente grande.

- Por este motivo, es imposible ejecutar todas las combinaciones de caminos durante las pruebas.
- Es posible, sin embargo, cubrir adecuadamente la lógica del programa y asegurarse de que se han aplicado todas las condiciones en el diseño a nivel de componente.
- Para ser más eficaces, las pruebas deberían ser realizadas por un equipo independiente.
- Por «mas eficaces» queremos referirnos a pruebas con la más alta probabilidad de encontrar errores (el objetivo principal de las pruebas).

Estrategias de prueba

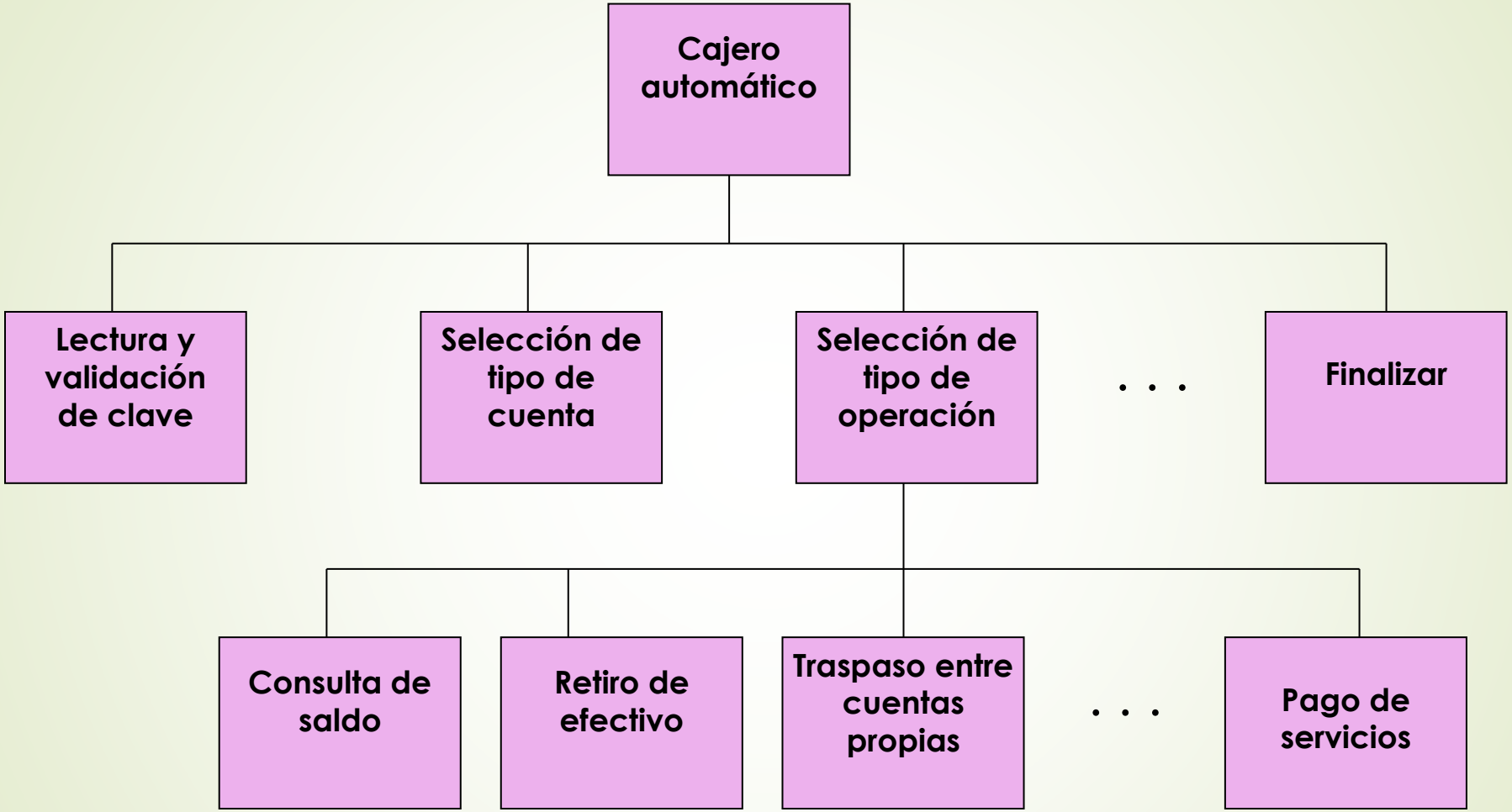
- Una estrategia de prueba del software integra las técnicas de diseño de casos de prueba en una serie de pasos bien planificados que dan como resultado una correcta construcción del software.
- Debe incorporar la planificación de la prueba, el diseño de casos de prueba, la ejecución de las pruebas y la evaluación de los datos resultantes.
- Debe abarcar pruebas de todos los segmentos de código fuente se han implementado correctamente, así como pruebas que validen las principales funciones del sistema frente a los requisitos del cliente.

Pruebas unitarias

- Una prueba unitaria es una forma de probar el correcto funcionamiento de un módulo de código.
- Esto sirve para asegurar que cada uno de los módulos funcione correctamente por separado.
- Las pruebas de unidad son un proceso para probar subprogramas, los procedimientos individuales o las clases de un programa.
- Es decir, es mejor probar primero los bloques desarrollados más pequeños del programa, que inicialmente probar el software en su totalidad.

Pruebas unitarias

- Facilita la eliminación de errores ya que al detectarlo se sabe en que módulo se ha producido.
- Se necesitan dos tipos de información al diseñar los casos de prueba para una prueba de unidad:
  - La especificación para el módulo donde se definen los parámetros de entrada y salida del módulo y su función.
  - El código fuente del módulo.


Cajero automático
Lectura y validación de clave
Selección de tipo de cuenta
Selección de tipo de operación
. . .
Finalizar
Consulta de saldo
Retiro de efectivo
Traspaso entre cuentas propias
. . .
Pago de servicios

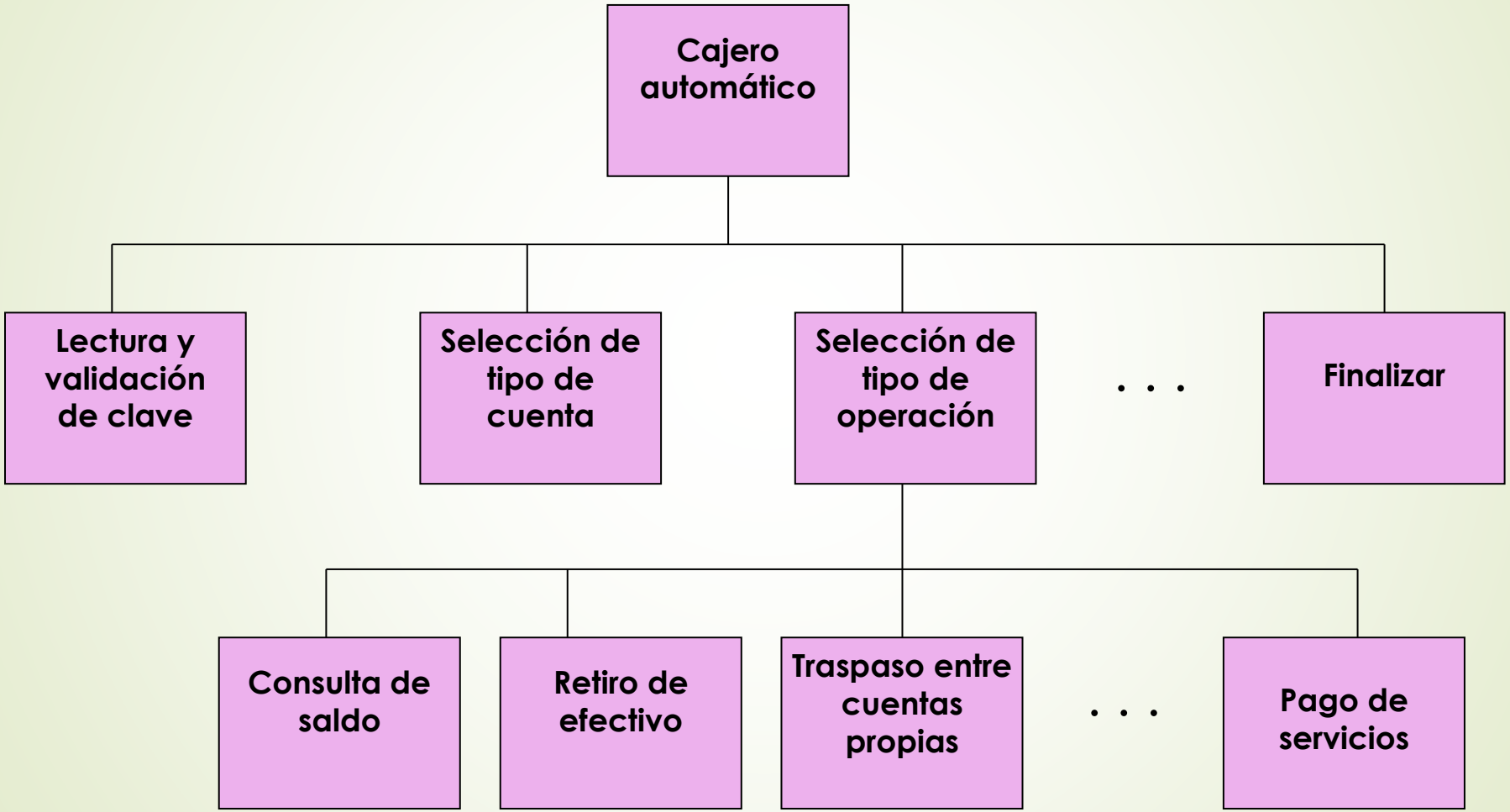
Pruebas de integración

- Las pruebas integrales o pruebas de integración son aquellas que se realizan una vez que se han realizado las pruebas unitarias.
- Consiste en realizar pruebas para verificar que un gran conjunto de partes de software funcionan juntos.
- Las pruebas de integración es la fase de la prueba de software en la cual los módulos individuales de software son combinados y probados como grupo.
- Las pruebas de integración son posteriores a las unitarias y anteriores a las de sistema.

Pruebas de integración

- **Integración incremental:** se combina el siguiente componente que se debe probar con el conjunto de componentes que ya están probados y se va incrementando progresivamente el número de componentes a probar.
- **Integración no incremental:** Se prueba cada componente por separado y posteriormente se integran todos de una vez realizando las pruebas pertinentes.

Pruebas unitarias VS Pruebas de integración



Pruebas de aceptación

- Las pruebas de aceptación o validación son el proceso de revisión mediante el cual se comprueba que el sistema de software producido cumple con las especificaciones y que cumplen su cometido.
- Se trata de evaluar el sistema o parte de este durante o al final del desarrollo para determinar si satisface los requisitos iniciales.
- La pregunta a responder es ¿Es esto lo que el cliente quiere?

Pruebas de aceptación

- Cuando se construye software a medida para un cliente, se lleva acabo una serie de pruebas de aceptación para permitir que el cliente valide todos los requisitos.
- Principalmente podemos distinguir entre pruebas alfa y beta.
- **Pruebas Alfa:** se llevan a cabo, por el cliente, en el lugar del desarrollo. El cliente utiliza el software de forma natural con el desarrollador observando al usuario y registrando los errores y problemas de uso.

Pruebas de aceptación

- Las pruebas alfa se llevan a cabo en un entrono controlado.
- **Pruebas Beta:** Se llevan a cabo por los usuarios finales del software en los lugares de trabajo de los clientes. A diferencia de la prueba alfa, el desarrollador no está presente.
- Así la prueba beta es una aplicación en vivo del software en un entorno que no puede ser controlado por el desarrollador. El cliente registra todos los problemas que encuentra durante la prueba beta e informa a intervalos regulares al desarrollador.

Pruebas de sistema

- Una aplicación software es sólo un elemento de un sistema mayor basado en una computadora.
- Es decir, que una vez acabada una aplicación, esta es incorporada a otros elementos del sistema (por ejemplo, a un determinado hardware).
- Por tanto es necesario realiza una serie de prueba de integración del sistema.
- La prueba del sistema está constituida por una serie de pruebas diferentes cuyo propósito primordial es ejercitar el sistema en su conjunto y no sólo la aplicación desarrollada.

Pruebas de sistema

- Aunque cada prueba tiene un propósito diferente, todas trabajan para verificar que se han integrado adecuadamente todos los elementos del sistema y que realizan las funciones apropiadas.

Tipos de pruebas de sistema:

- **Prueba de recuperación**: Muchos sistemas o aplicaciones deben recuperarse de fallos y continuar el proceso en un tiempo previamente especificado y sin perdida de datos. **Ejemplo: Corte de luz**.

Pruebas de sistema

- Una prueba de recuperación es una prueba del sistema que fuerza el fallo del software de muchas formas y verifica que la recuperación se lleva acabo apropiadamente:
- Si la recuperación es automática hay que evaluar la corrección de la inicialización, la recuperación de los datos y el proceso de rearranque.
- Si la recuperación requiere la intervención humana, hay que evaluar los tiempos medios de reparación para determinar si están dentro de unos límites aceptables.

Pruebas de sistema

- **Prueba de seguridad:** Cualquier sistema basado en computadora que maneje información sensible o lleve a cabo acciones que puedan perjudicar (o beneficiar) impropiamente a las personas será un objetivo de ataque por “hackers” por deporte o para obtener ganancias ilícitas.
- La prueba de seguridad intenta verificar que los mecanismos de protección incorporados en el sistema lo protegerán de accesos inapropiados.
- El responsable de la prueba desempeña el papel de un individuo que desea entrar en el sistema.

Pruebas de sistema

- Su labor será intentar entrar al sistema de cualquier forma.
- El único sistema que es totalmente seguro es aquel que se encuentra apagado y desconectado, guardado en una caja fuerte de titanio que está enterrada en cemento, rodeada de gas nervioso en un bunker en medio del desierto vigilado por un grupo de guardias fuertemente armados y muy bien pagados. Aún así, no apostaría mi vida en ello“ (Gene Spafford).

Pruebas de sistema

- **Prueba de resistencia:** Las pruebas de resistencia están diseñadas para enfrentar a los programas con situaciones anómalas.
- En esencia , el sujeto que realiza la prueba de resistencia se pregunta: ¿A qué potencia puedo poner a funcionar el sistema antes de que falle?
- La prueba de resistencia ejecuta un sistema de forma que demande recursos en cantidad, frecuencia o volúmenes anormales.

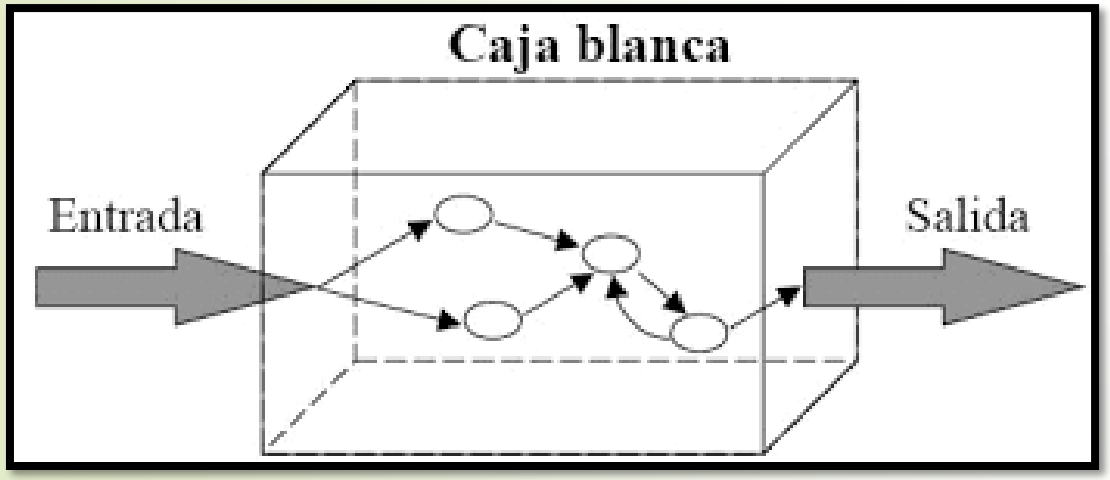
Pruebas de sistema

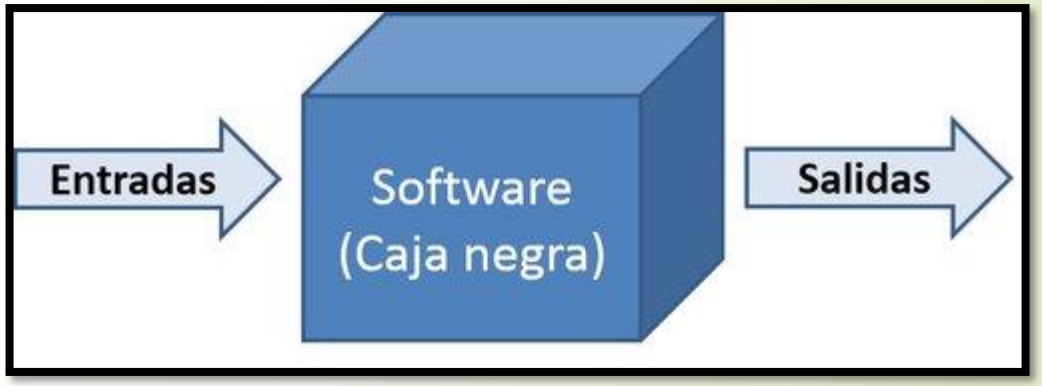
- Por ejemplo: ejecutar casos de prueba que requieran el máximo de memoria o diseñar casos de prueba que produzcan excesivas búsquedas de datos residentes en disco.
- En definitiva el responsable de la prueba intenta romper el programa.
- **Prueba de rendimiento**: Son las pruebas que se realizan para determinar si un sistema realiza una tarea de forma correcta y en el menor tiempo posible en una condiciones particulares de trabajo.

Pruebas de sistema

- Las pruebas de rendimiento pueden servir para diferentes propósitos.
- Pueden demostrar que el sistema cumple los criterios de rendimiento.
- En dichos sistemas es inaceptable que el software no se ajuste a los requisitos exigidos cuando el sistema se somete a condiciones de prueba que sean similares a las esperadas en el uso real.

- Existen pruebas de **caja negra** que son pruebas sobre la interfaz del software y su operatividad.
- Existen pruebas de **caja blanca** que son pruebas minuciosas que examinan los detalles del código fuente.





- Vamos a hacer un ejemplo de pruebas de caja negra.
- Para ello tenemos la clase Calculadora que contiene una serie de métodos que realizan una serie de operaciones matemáticas.

```
public class Calculadora {
    public static int suma(int a,int b){
        return a+b;
    }
    public static int resta(int a,int b){
        return a-b;
    }
    //...
}
```

- En el método main realizaríamos las pruebas del software:

```java
public static void main(String[] args){
    int res=Calculadora.suma(4, 5);

    if(res==9){
        System.out.println("Prueba superada");
    }else{
        System.out.println("Prueba no superada");
    }
}
```

- Es bastante engorroso porque mezclamos el código de distintas pruebas.
- Una solución es usar JUnit que es un plugin que permite automatizar las pruebas en paralelo y de manera mucho más cómoda.
- Proporciona software que realiza directamente las comparaciones e indica de manera visual si el resultado devuelto por la función es el esperado o no.

- Es muy importante incluir @Test antes del método porque es una advertencia a JUnit para que entienda que es un posible método de prueba.
- Para poder usar un método de una clase fuera de ella hay que poner el nombre de la misma.
- El método assertEquals comprueba que los valores son iguales mostrando si la prueba consigue el valor esperado o no.
- Lo que queda es pulsar el botón derecho encima del código del método para ver el resultado de la prueba de caja negra.

Bibliografía

- **Braude, E.J.: "Ingeniería del Software: una perspectiva orientada a objetos".** Editorial RA-MA. 2003
- **García de Jalón, j.: "Aprende Java como si estuvieras en primero".** Editorial TECNUN. 2000
- **Beizer, B., Software Testing Techniques, 2.a ed.,** Editorial Van Nostrand Reinhold
- **Casado Iglesias, C.: "C.F.G.S Entornos de desarrollo"** Editorial RA-MA.
- **Wikipedia, la enciclopedia libre. http://es.wikipedia.org/** Última visita: Octubre 2018.
- **López, J.C.: "Curso de JAVA http://www.cursodejava.com.mx** Última visita: Octubre 2018**.**
- **Documentación oficial Java JSE 8 http://docs.oracle.com/javase/8/** Última visita: Octubre 2015.
- **Programación en castellano: Java. http://www.programacion.net/java Última visita: Octubre 2015.**